

5

Linear Classification

The previous chapter addressed linear regression. What about linear classification? In classification f should return a discrete value, one of a fixed finite number of categories, or classes. So, f cannot be a linear function of its inputs because its output is not a continuous value.

However, this does not mean we cannot use linear functions to construct a classifier. Linear functions make great building blocks for many machine learning methods because of their simplicity. In this chapter we will explore linear methods for classification, focusing on binary classification (classification where there are only two possible outputs). We begin with how to describe a binary classifier. Later, we look at how to learn the parameters of that classifier from data.

Binary Classification

We will first consider the most basic (and most common) classification task: determining whether the input belongs to one of two classes. This is called **binary classification**. Whether the classes be cancer and non-cancer (for cell classification), normal or defective (for assembly line quality control), or face and not-face (for face detection in images), mathematically we often represent the classes as $+1$ and -1 . The positive class (associated with $+1$) typically represents the class “of interest.” For example, in disease detection, the positive class is typically the presence of the disease, ironically. Table 5.1 shows one example dataset. The goal is to generate a rule that will determine whether a hippopotamus is aggressive or not, based on its girth and ear wiggle rate.

If f cannot be linear (as its output should be either $+1$ or -1 , not a general real value), what can be a linear function? For binary classification, it is common to learn a **discriminant**: a function whose sign matches that of the output. We will use $\tilde{f}$ to denote this function.

Further information

It is also common to use $+1$ and 0 to denote the two classes, still often called the positive and negative classes confusingly. For the exposition in this chapter, $+1$ and -1 are more natural, so we will stick with them.

Further information

Often, f itself is used to denote the discriminant, because the discriminant and the prediction are so similar. However, this can be confusing when first learning, so we will use separate notation for them.

girth	ear wiggle rate	aggressive		
3.49	0.41	no	$X =$	$Y =$
2.39	0.34	no		
3.28	0.14	yes		
3.10	0.31	yes		
3.01	0.27	no		
3.11	0.53	yes		
2.57	0.53	yes		
3.07	0.38	yes		
2.54	0.36	no		
3.64	0.05	no		

Table 5.1: Hippopotamus aggression data

Thus

$$f(\vec{x}) = \text{sgn}(\tilde{f}(\vec{x})) \quad (5.1)$$

where sgn is the sign of its argument as either $+1$ or -1 . That is, when $\tilde{f}(\vec{x})$ is positive, $f(\vec{x})$ is $+1$, and when $\tilde{f}(\vec{x})$ is negative, $f(\vec{x})$ is -1 . We will make this discriminant function a linear function of x . The sign of its output is the predicted class.

Further information

When $\tilde{f}(\vec{x})$ is exactly equal to 0, we can pick either $+1$ or -1 .

Decision Boundary

For any classification method (whether it uses discriminants or not), we can define the **decision boundary** as the set of inputs exactly between those that are classified as one class and those that are classified as a different class. Thus, if $\vec{x}$ is on a decision boundary, moving it slightly in any direction (except along the decision boundary) will cause f to classify the point as one class, and moving it slightly in the opposite direction will cause f to classify the point as a different class. The decision boundary is the set of inputs for which f cannot choose between the two classes.

If our f is expressed as in Equation 5.1, this boundary corresponds to the set of points for which the discriminant is equal to 0: $\tilde{f}(\vec{x}) = 0$. In this chapter, we are considering linear discriminants, and so

$$\tilde{f}(\vec{x}) = \vec{x}^T \vec{w} \quad (5.2)$$

where, just as in linear regression, we have augmented $\vec{x}$ with an extra component of 1 and included an extra component in $\vec{w}$ to account for the intercept term (see Equation 4.2).

Thus, for binary classification, using a linear discriminant, the decision boundary is the set of points $\vec{x}$ for which

$$0 = \vec{x}^T \vec{w} . \quad (5.3)$$

This is the description of a linear surface. If $\vec{x}$ is in two-dimensional space, this is the equation for a line:

$$\begin{aligned} 0 &= w_0 + w_1 x_1 + w_2 x_2 \\ x_2 &= \frac{-w_1}{w_2} x_1 + \frac{-w_0}{w_2} \quad \text{if } w_2 \neq 0 \end{aligned} \quad (5.4)$$

(showing it in its common “ $mx + b$ ” form). If $\vec{x}$ is in three-dimensional space, Equation 5.3 is the equation for a plane. In general, if $\vec{x}$ is in n dimensions (there are n features), this is the formula for an $(n-1)$ -dimensional hyperplane.

Therefore, using a linear discriminant produces a linear decision boundary for binary classification. The discriminant is *not* the decision boundary. The decision boundary is where the discriminant is equal to 0, that is, where it intersects a horizontal plane at “ $y = 0$.”

The vector $\vec{w}$, excluding the w_0 component, is orthogonal to (also known as “normal to”) the decision boundary. The offset, w_0 , controls the position of the decision boundary along the direction of $\vec{w}$. Figure 5.2 illustrates this.

Perceptron Learning Algorithm

Combining Equations 5.1 and 5.2 we have a predictor often known as a **perceptron**:

$$f(\vec{x}) = \text{sgn}(\vec{x}^T \vec{w}) . \quad (5.5)$$

This classifier takes the inner product of the input with its weight vector (and adds a constant) and then classifies based on the sign of the output. This is a very simple function, but it can work very well for many problems. We can see it as assigning a weight to each feature. A positive weight for a feature indicates that larger values of that feature are more indicative of the positive class. A negative weight indicates that larger values are more associated with the negative class. For feature i , we can view the inner product term $x_i w_i$ as a score for that feature. The total score is the sum over every feature. The offset, w_0 , sets a threshold of $-w_0$ for how large the resulting total score must be for the example to be classified as the positive class.

Further information

To reiterate, a weight does not indicate any causal relationship between the feature and the class, only that the feature is either positively or negatively related (in some way) to the class when considered with all other features.

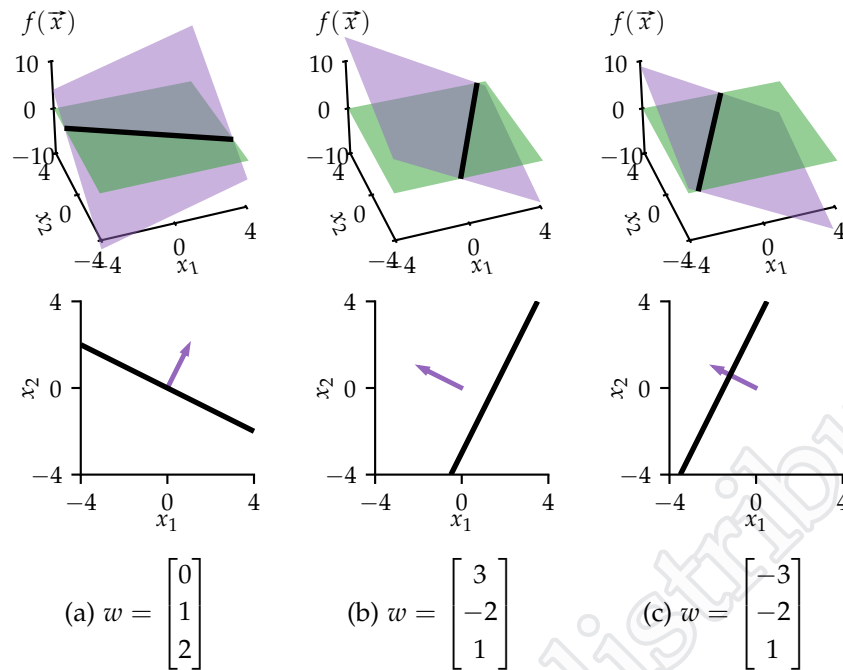


Figure 5.2: Examples of decision boundaries from linear discriminant functions.

(Top) Diagonal plane is the discriminant function; Horizontal plane is $y=0$; their intersection is the decision boundary

(Middle) Line is the decision boundary; vector is the weight vector (except for zeroth, intercept component) plotted from the origin

(Bottom) Weight vectors (a) No intercept, decision surface is orthogonal to weight vector, (b) Positive intercept, decision surface is orthogonal to weight vector, but shifted in the opposite direction of the weight vector because the discriminant is shifted upward, (c) Negative intercept, same as positive intercept, but decision surface is shifted in the same direction as the weight vector.

Perceptron Learning Algorithm

So how to learn the parameters, $\vec{w}$, from a dataset? Example i from the dataset is correctly classified if y_i has the same sign as $\tilde{f}(\vec{x}_i) = \vec{x}_i^\top \vec{w}$. Because y_i is either $+1$ or -1 , this is the same as saying that example i is correctly classified if $y_i \vec{x}_i^\top \vec{w} > 0$. Thus, we would like $\vec{w}$ to point roughly in the same direction as (that is, within 90° of) the vector $y_i \vec{x}_i$.

This is the essence behind the perceptron learning algorithm. We cycle through each point in the dataset individually. For each point, we check to see if we are currently classifying the point correctly. If we are, we do not adjust the weight vector. If we are not, we add the vector of the point times its class label to the weight vector, to move the weight vector in that direction. That is, if the point has a positive label, we add its feature vector to the weights; and if it has

PERCEPTRON LEARNING

In: X, Y

Out: $\vec{w}$

$\vec{w} \leftarrow \vec{0}$

while not done **do**

for all $i \in \{1, \dots, m\}$ **do**

if $y_i \vec{x}_i^\top \vec{w} \leq 0$ **then**

$\vec{w} \leftarrow \vec{w} + y_i \vec{x}_i$

return $\vec{w}$

Algorithm 5.3: Perceptron learning algorithm

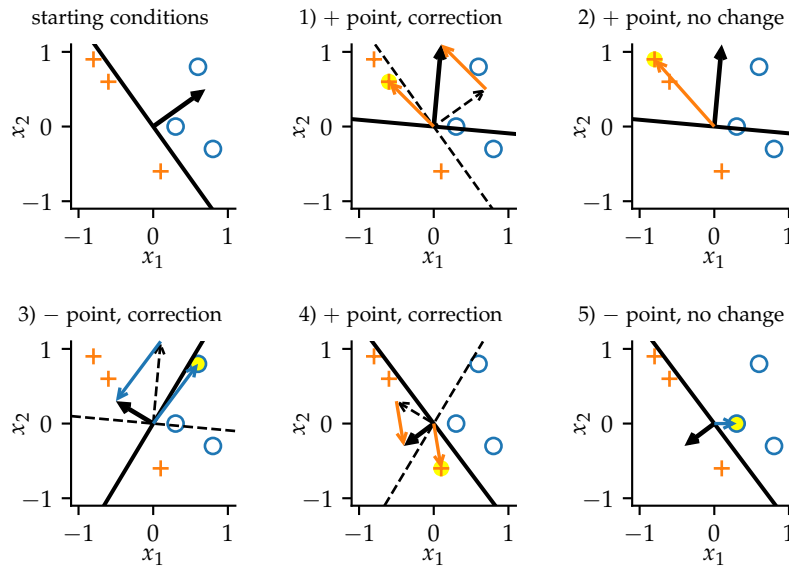


Figure 5.4: A few steps of the perceptron learning algorithm, starting with a non-zero w and not using an intercept term.

Orange +s are positive points. Blue Os are negative points.
 Bold arrow and line are the updated weight vector and decision boundary.
 Dashed arrow and line are the previous vector and boundary.
 Colored arrow is vector to current point considered.
 Text shows iteration number and what occurred.

a negative label, we subtract its feature vector from the weights. We then continue with the next point.

The perceptron learning algorithm is shown in Algorithm 5.3. The starting point for this algorithm ($\vec{w} = \vec{0}$) is common, but any starting vector will work. The loop “while not done” continues until no changes are made during an entire sweep of all of the points (the inner “for all” loop). An example of how the algorithm runs is shown in Figure 5.4. After the fifth iteration, the algorithm classifies all six training points correctly. It will run once more through all of the points, noting it makes no changes, and then quit. In this example, each change fixed the examined example. That will not always be the case.

If the data are **linearly separable** (there exists some line, plane, or hyper-plane that separates the negative points from the positive points), then the perceptron learning algorithm will stop and return a weight vector that classifies all of the training data correctly.

However, if no such separating decision surface exists, the al-

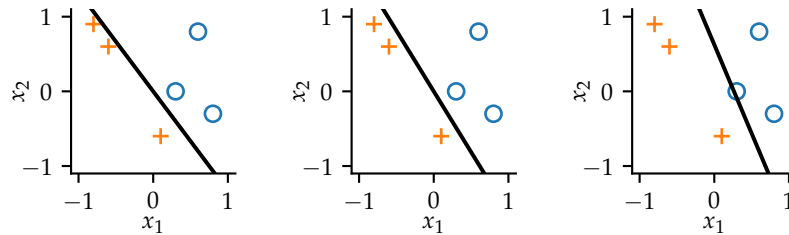


Figure 5.5: Multiple possible separating boundaries for the same data

gorithm will never halt. There is no straight-forward way of determining in the middle of the algorithm whether continuing for a while longer will result in the algorithm stopping. Further, if the data are separable, there are many lines (or planes or hyper-planes) that would separate the data (see Figure 5.5). The perceptron learning algorithm selects a mostly arbitrary one. While it may get all of the training data correct, this does not mean that it is the best choice for the future predictions (the testing data). For these reasons, other methods (such as the one discussed next) are preferred.

Further information

The perceptron learning algorithm is the only algorithm presented in this text that is not commonly used in modern machine learning. However, it serves as a good introduction to linear classification methods, motivates our next method well, and is of historic importance.

Logistic Regression Model

While there are multiple linear decision boundaries that can correctly classify all examples in some training datasets, we would not consider them all equal. In Figure 5.5, most would prefer the middle boundary to the other two because it is more likely to correctly classify future points, particularly those near the boundary. Yet, the perceptron learning algorithm would pick one arbitrarily. Further, if there are no separating boundaries, the perceptron learning algorithm will continue forever, essentially unable to select a boundary, despite that there may be some with relatively few errors.

We would like a method that solves both of these problems. One hint at a solution is with the form of the perceptron classifier in Equation 5.5. The result, $f(\vec{x})$, is invariant to positive scaling of $\vec{w}$. That is, if we multiply all elements in $\vec{w}$ by the same positive amount, the decision surface does not change.

Therefore, we have an extra degree of freedom that is not doing anything useful. It is the steepness of the function $\tilde{f}(\vec{x})$ but does not affect $\text{sgn}(\tilde{f}(\vec{x}))$ or where $\tilde{f}(\vec{x}) = 0$. Thus, while the perceptron learning algorithm only cares about the sign of $\tilde{f}(\vec{x})$, this new method will also use the magnitude of $\tilde{f}(\vec{x})$.

Further information

For example, $2 + 3x_1 - x_2 = 0$ and $4 + 6x_1 - 2x_2 = 0$ represent the same line.

Log-Odds and the Sigmoid

We will let a more positive value of $\tilde{f}(\vec{x})$ correspond to a greater chance that the example is from the positive class and a more negative value of $\tilde{f}(\vec{x})$ correspond to a greater chance that the example is from the negative class. The discriminant is on the range $(-\infty, +\infty)$ and the probability of the positive class is on the range $[0, 1]$, so we will need a mapping.

The **log-odds** is particularly well suited for this. That is, we will associate $\tilde{f}(\vec{x})$ with the log-odds of the positive class, conditioned on the point $\vec{x}$:

$$\tilde{f}(\vec{x}) = \ln \left(\frac{\Pr(y = +1 | \vec{x})}{1 - \Pr(y = +1 | \vec{x})} \right). \quad (5.6)$$

For example, for points $\vec{x}$ for which $\tilde{f}(\vec{x}) = 2$ we expect the log-odds of these points being of the positive class to be 2. Or, we expect that 88% of such points would be of the positive class: $2 = \ln \frac{0.88}{0.12}$. As another example, when $\tilde{f}(\vec{x}) = -1$ we expect that 17% of the time, the point would be of the positive class. See Figure 5.6.

Reworking Equation 5.6 we get

$$\begin{aligned} \Pr(y = +1 | \vec{x}) &= \frac{1}{1 + e^{-\tilde{f}(\vec{x})}} \\ &= \sigma(\tilde{f}(\vec{x})) \end{aligned} \quad (5.7)$$

where we have defined

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad (5.8)$$

The function σ (plotted in Figure 5.7) is the inverse of the log-odds function and is known as the **sigmoid** function or **logistic** function. It is related to the hyperbolic tangent: $\tanh z = 2\sigma(2z) - 1$. While interesting, we do not use this relationship in this text.

We will require the derivative of the sigmoid function:

$$\begin{aligned} \frac{d\sigma(z)}{dz} &= - \left(\frac{1}{1 + e^{-z}} \right)^2 (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \\ &= \frac{1}{1 + e^{-z}} \frac{e^{-z}}{1 + e^{-z}} \\ &= \frac{1}{1 + e^{-z}} \left(1 - \frac{1}{1 + e^{-z}} \right) \\ &= \sigma(z) (1 - \sigma(z)). \end{aligned} \quad \begin{array}{l} \text{a reasonable place to stop} \\ \text{a cute place to stop} \end{array} \quad (5.9)$$

That the derivative of the sigmoid function can be written in terms of itself is not special. For example, the derivative of e^{2x} can be written as twice the original function. The main purpose for this is to

Key point

If p is the probability of an event, the log-odds of the same event are $\ln \left(\frac{p}{1-p} \right)$.

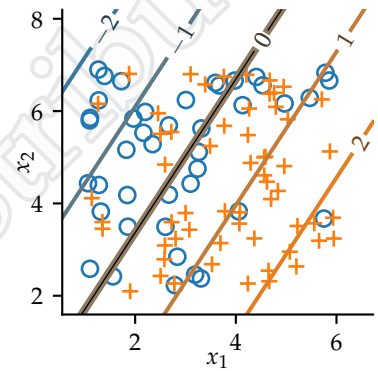


Figure 5.6: Linear discriminant fit to data. Orange +s are positive points. Blue Os are negative points. The iso-contours of the discriminant are shown.

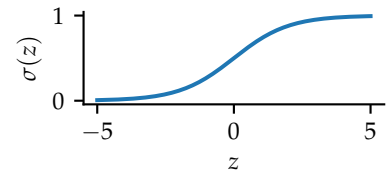


Figure 5.7: The sigmoid

make explicit how we can save on computation. If we have already calculated $\sigma(z)$, we can calculate its derivative without much extra effort.

One other useful fact: $\sigma(-z) = 1 - \sigma(z)$. Thus

$$\begin{aligned}\Pr(y = +1 \mid \vec{x}) &= \sigma(\tilde{f}(\vec{x})) \\ \Pr(y = -1 \mid \vec{x}) &= 1 - \sigma(\tilde{f}(\vec{x})) = \sigma(-\tilde{f}(\vec{x}))\end{aligned}$$

which means that

$$\Pr(y = c \mid \vec{x}) = \sigma(c\tilde{f}(\vec{x})) \quad c \text{ is either } +1 \text{ or } -1$$

which is often just written as

$$\Pr(y \mid \vec{x}) = \sigma(y\tilde{f}(\vec{x})) . \quad (5.10)$$

Conditional Log-Likelihood

Adopting this interpretation of $\tilde{f}(\vec{x})$ as the log-odds of the positive class for an example with features of $\vec{x}$, we will define the goal of our learning method. In particular, we will seek to tune $\tilde{f}(\vec{x})$ by changing the weights, $\vec{w}$, inside $\tilde{f}$ to maximize the conditional likelihood of the training data, or $\Pr(\mathbf{Y} \mid \mathbf{X})$. This is the probability that each of the training labels are the values we observed, given the features associated with those examples. It can be rewritten as

$$\begin{aligned}\Pr(\mathbf{Y} \mid \mathbf{X}) &= \prod_{i=1}^m \Pr(y_i \mid \vec{x}_i) \quad \text{each example is drawn independently} \\ &= \prod_{i=1}^m \sigma(y_i \tilde{f}(\vec{x}_i)) \quad \text{from Equation 5.10} \\ &= \prod_{i=1}^m \sigma(y_i \vec{x}_i^\top \vec{w}) . \quad (5.11)\end{aligned}$$

This expression is in terms of the data $(\vec{x}_1, y_1, \vec{x}_2, y_2, \dots, \vec{x}_m, y_m)$ and the weights of our classifier ($\vec{w}$). We wish to select the $\vec{w}$ that maximizes this value. For computational and notational ease, we will transform it into an equivalent problem. First, the $\vec{w}$ that maximizes $\Pr(\mathbf{Y} \mid \mathbf{X})$ is the same as the $\vec{w}$ that maximizes $\ln \Pr(\mathbf{Y} \mid \mathbf{X})$ because $\ln$ is a monotonically increasing function: If $a > b$ then $\ln a > \ln b$. Second, instead of maximizing $\ln \Pr(\mathbf{Y} \mid \mathbf{X})$, we will minimize $-\ln \Pr(\mathbf{Y} \mid \mathbf{X})$. This second is a trivial change, but couches the optimization as a minimization (instead of a maximization) which is more standard.

Further information

In the notation $\Pr(y = c \mid \vec{x})$, y is a random variable and c is a value for that random variable. In the notation $\Pr(y \mid \vec{x})$, y is the value of an (implied) random variable. It is common (although occasionally confusing) to use the same letter to denote a random variable and a value for that random variable. As with most communication, a balance in clarity must be struck between being too verbose (spelling everything out, but risking losing the thread of what is important) and being too terse (demonstrating the essential point, but risking confusion over the meaning).

Because we seek to minimize our objective, we call it a loss, L :

$$\begin{aligned}
 L &= -\ln \Pr(\mathbf{Y} \mid \mathbf{X}) \\
 &= -\ln \prod_{i=1}^m \sigma(y_i \vec{x}_i^\top \vec{w}) \\
 &= \sum_{i=1}^m -\ln \sigma(y_i \vec{x}_i^\top \vec{w}) \\
 &= \sum_{i=1}^m l(y_i, \vec{x}_i^\top \vec{w})
 \end{aligned} \tag{5.12}$$

where the per-item loss is

$$l(y, \tilde{f}) = -\ln \sigma(y \tilde{f}) \tag{5.13}$$

The objective L is known as the negative (conditional) log-likelihood¹. Just as the loss for least squares of Equation 4.4, this loss breaks down into a sum over a per-item loss evaluated on each example. This per-item loss, l , is a function of the true label, y , and the prediction from our discriminant function, $\tilde{f}$.

In the perceptron learning algorithm, we knew we wanted $y_i \vec{x}_i^\top \vec{w} > 0$ for all i , thus making every example properly classified. However, if this could not be achieved, we did not have a method for balancing errors across the examples to get a single set of weights. Equation 5.12 achieves this. It looks at $y_i \vec{x}_i^\top \vec{w}$, just as the perceptron does. But, instead of merely checking to see if it is positive, it scores this scalar value as $-\ln \sigma(y_i \vec{x}_i^\top \vec{w})$ and adds these scores across all examples to judge a particular discrimination function. For a given example, if $y_i \vec{x}_i^\top \vec{w}$ is positive and large, the score will be small and near zero. If it is very negative, the score will be very large.

While we could have combined the discriminant's value for each example in any number of ways, this approach takes a principled way of finding a total loss based on maximizing the likelihood and treating the discriminant function as the log-odds. This is called **logistic regression** because it predicts the log-odds using the logistic function (or sigmoid function).

Ideally at this point, we would proceed as we did for linear regression: take the gradient of the loss with respect to $\vec{w}$, set it equal to $\vec{0}$, and manipulate the resulting equation to get $\vec{w}$ in terms of the data. However, the resulting equation cannot be so manipulated. Therefore, we cannot write down a closed-form or analytic expression for $\vec{w}$. We must revert to using numeric optimization to find the value of $\vec{w}$ that minimizes the loss. And for that, we must take a detour.

Further information

This per-item loss function is often known as the **logistic loss**.

¹ Why all these modifiers?

negative: negative sign in front

conditional: conditioned on $\mathbf{X}$, although sometimes omitted from the name

log: logarithm

likelihood: probability of the data under the model

Key point

Logistic regression, despite its name, is a *classification* method.

Gradient Descent

Numeric optimization is the study of how to optimize (find the max-

imum or minimum of) a function using computational algorithms. Generally, we assume that we can evaluate the function at any point of interest and possibly evaluate other aspects of the function at that point (for example, its derivative). The field is very large, addresses many variants of the problem, and has all kinds of algorithms. It is critical for machine learning, as most machine learning methods result in an optimization problem to be solved. For this text, we will only cover the basics of one method, gradient descent. It is the most commonly used in machine learning and sufficient for us.

Gradient descent assumes there is a function whose input argument is a vector and whose output is a scalar. Further, we assume that for any input, we can calculate the value of the function and the gradient of the function at that input. However, we do not assume we know anything else about the function. Our goal is to query the function (and its gradient) repeatedly in order to find the input that minimizes the function's output. While it is common to use x to denote the input and f to denote the function, for machine learning this can be confusing because x is a data point and f is a predictor; however, we do not use gradient descent to optimize the data and the function of interest is not the prediction function directly. Therefore, in this text we will minimize $L(\vec{w})$ with respect to the input, $\vec{w}$. This matches its use in machine learning to find a weight vector that minimizes the loss. The technique is more general, though.

The essential insight for gradient descent is that the gradient vector points in the direction that is (locally) most “uphill.” And, therefore, moving in the opposite direction will help to minimize the function. Figure 5.8 illustrates this for a function of two dimensions. The gradient is perpendicular to the iso-contour (surface of points that have the same function value) at the same point. These iso-contours are one-dimensional curves in this example, but if $\vec{w}$ were of higher dimension, they would be surfaces or hypersurfaces. The gradient is a vector in the same dimension of $\vec{w}$ and therefore is shown lying “flat” in the w_1 - w_2 plane.

Gradient descent proceeds by taking small (but finite) steps in the opposite direction of the gradient. Because the gradient changes with $\vec{w}$, the gradient is recomputed after each step. A sketch of the algorithm is in Algorithm 5.9. Three points of this algorithm are vague: 1. the starting point, 2. how to check convergence, and 3. how to select η . We will address them in the reverse order.

The scalar η is called the **step size**. As presented here, it is a constant. As η approaches 0, the algorithm moves the point like a ball rolling down a hill would, if there were a lot of friction. Therefore, if the step size is small enough, the algorithm will converge to a *local* minimum of L . However, the smaller the step size, the more steps

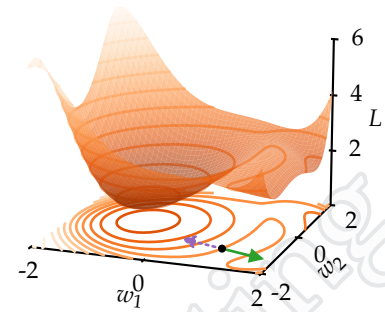


Figure 5.8: A gradient example. The function is the upper surface. The gradient at the black point is the solid green vector; its negative is the dotted violet vector.

GRADIENT DESCENT LOSS MINIMIZATION

In: $\mathbf{X}, \mathbf{Y}, \eta$

Out: $\vec{w}$

$\vec{w} \leftarrow$ arbitrary point

while not converged **do**

$\vec{w} \leftarrow \vec{w} - \eta \nabla L(\vec{w})$

return $\vec{w}$

Algorithm 5.9: Gradient descent algorithm

must be taken to reach convergence and the longer the algorithm will take computationally. If the step size is too large, though, the algorithm can fail to converge, essentially jumping across minima and often diverging. How small is small enough depends on properties of the function L (like how steep it can be).

While sometimes a fixed value, like 0.1, is chosen for η , more commonly η is changed as the algorithm proceeds. The simplest method is to reduce η on each iteration. If η_k is the value on iteration k , a simple method like $\eta_k = 1/k$ will guarantee convergence. However, it can be very slow in practice. More often, an adaptive method is used where η is changed based on the gradients computed: increased when consecutive steps are in the same direction and decreased when consecutive steps lead in opposite directions.

Convergence occurs when the gradient is equal to $\vec{0}$: At that point further updates inside the while-loop make no changes to $\vec{w}$. This algorithm is almost always implemented in floating point arithmetic and so, due to the finite precision of the operations, the gradient is often never exactly equal to zero. Further, we may not want to wait for the exact minimum if a solution very close to the minimum is possible much faster. Therefore, we often check whether the length of $\nabla L(\vec{w})$ is close to zero (less than a specified threshold). Of course, the gradient being near zero does not mean we are near a stationary point. There might be a long near-plateau the algorithm needs to traverse before finding a valley that is much better. We can also check the improvement in the objective, $L(\vec{w})$, from one iteration to the next and stop if it is less than a specified threshold.

If the algorithm is stopped when the gradient is zero (or the point stops moving), the result will be a stationary point (a maximum, a minimum, or a saddle point) of the function. While theoretically possible to converge to a maximum or saddle point, this is in practice very rare and we do not worry about this possibility. The gradients near a maximum or saddle point lead away from it, so the algorithm would need to accidentally land right on the point for it to stop. Therefore, in practice, gradient descent converges to a minimum, but a *local* minimum. There is no guarantee that we will find the global minimum. Indeed, without external knowledge of the properties of the function, we can never guarantee there is not another minimum in an unsearched part of the input space. We take this as the price for the ability to optimize an arbitrary function with only the ability to evaluate its gradient.

Because the algorithm finds a local minimum, the point returned can depend on the starting point. Ideally, we would start the algorithm near the global minimum, so it can “fall” right into it. However, we often do not know such a starting point. We might try a

simple starting point, like $\vec{w} = \vec{0}$. Although this initialization is fine for the methods in this chapter, for many situations it can be problematic because of symmetries in L that cause this to be exactly a maximum or saddle point. Therefore, we often select a random point (from a bounded range of reasonable starting points). Multiple random points can also be tried to see which produces the best local minimum, at the price of increased computation time.

Logistic Regression Learning

With that important digression out of the way, we return to logistic regression learning. We apply the gradient descent algorithm to perform the optimization to minimize the logistic regression loss function, Equation 5.12. To do so, we need to be able to calculate the gradient of the loss:

$$\begin{aligned}
 \nabla_{\vec{w}} L &= \nabla_{\vec{w}} \sum_{i=1}^m -\ln \sigma(y_i \vec{x}_i^T \vec{w}) \\
 &= \sum_{i=1}^m \frac{-1}{\sigma(y_i \vec{x}_i^T \vec{w})} \nabla_{\vec{w}} \sigma(y_i \vec{x}_i^T \vec{w}) && \text{derivative of sum and chain rule} \\
 &= \sum_{i=1}^m \frac{-1}{\sigma(y_i \vec{x}_i^T \vec{w})} \sigma(y_i \vec{x}_i^T \vec{w}) (1 - \sigma(y_i \vec{x}_i^T \vec{w})) \nabla_{\vec{w}} (y_i \vec{x}_i^T \vec{w}) && \text{chain rule, see Equation 5.9} \\
 &= \sum_{i=1}^m \frac{-1}{\sigma(y_i \vec{x}_i^T \vec{w})} \sigma(y_i \vec{x}_i^T \vec{w}) (1 - \sigma(y_i \vec{x}_i^T \vec{w})) y_i \vec{x}_i \\
 &= - \sum_{i=1}^m (1 - \sigma(y_i \vec{x}_i^T \vec{w})) y_i \vec{x}_i \tag{5.14}
 \end{aligned}$$

We can calculate this with a loop through each example. Plugging in this formula into the gradient descent algorithm yields a logistic regression learning algorithm, Algorithm 5.10.

The inner loop calculates the sum of Equation 5.14. Because the loss is a sum over every training point, the gradient is also a sum over every training point. The sum is *added* to $\vec{w}$, because gradient descent subtracts the gradient, but the gradient is $-\vec{s}$.

Minima

The logistic regression loss is convex.² This does not quite imply that there is a single minimum. It could be that the loss function continues to decrease in one direction (like e^x continues to get smaller for smaller and smaller x). Then there is no minimum. Or, it could be that there is a direction in which the function is flat (like $(x - y)^2$ is flat along the line $x = y$). Then there are a set of minima, all connected and with the same loss.

LOGISTIC REGRESSION LEARNING

In: X, Y, η

Out: $\vec{w}$

```

 $\vec{w} \leftarrow$  arbitrary point
while not converged do
   $\vec{s} \leftarrow \vec{0}$ 
  for all  $i \in \{1, \dots, m\}$  do
     $\vec{s} \leftarrow \vec{s} + (1 - \sigma(y_i \vec{x}_i^T \vec{w})) y_i \vec{x}_i$ 
   $\vec{w} \leftarrow \vec{w} + \eta \vec{s}$ 
return  $\vec{w}$ 

```

Algorithm 5.10: Gradient descent logistic regression learning algorithm

² See Exercise 5.5.

If the data are linearly separable, there is no minima (or, we can view it as being “off at infinity” in some direction). If we take any $\vec{w}$ that separates the data, we can multiply it by 2 (or any scalar greater than 1) and end up with a $\vec{w}$ with a better loss. Consider Equation 5.12, reproduced here:

$$L = \sum_{i=1}^m -\ln \sigma(y_i \vec{x}_i^T \vec{w}) . \quad (5.15)$$

If the data are currently separated by $\vec{w}$, then $y_i \vec{x}_i^T \vec{w} > 0$ for all i . Therefore, multiplying $\vec{w}$ by 2 will make all of the $y_i \vec{x}_i^T \vec{w}$ terms bigger which will make all of the $-\ln \sigma(y_i \vec{x}_i^T \vec{w})$ terms smaller, which will make the sum smaller, and make the loss better. So, we can always improve any given separating weight vector.

This is in stark contrast to the perceptron learning algorithm where it only finished when the data were linearly separable. Algorithm 5.10 will only converge if the data are *not* separable (or if we terminate the gradient descent before the gradient becomes exactly zero). While this might seem like a problem, we can fix it easily with a bit of regularization, which we will cover in the next chapter.

If the data have linearly redundant features (such as the features of the hippopotamus’s height, the hippopotamus’s length, and the sum of the hippopotamus’s height and length), then the loss function will have a set of minima. This is not generally a problem, as gradient descent will terminate. But, you must be aware that the solution is not unique; it is only equivalent to any other solution in that it has the same loss.

Otherwise, if the data are not linearly separable and the features are not linearly dependent, then Algorithm 5.10 will terminate with the unique solution (at least up to the numeric tolerance specific for the termination condition) if η is handled properly (see previous section on selecting the step size).

Online Updates

Algorithm 5.10 calculates the gradient of the per-item loss for each example in the inner loop. Only once all of these have been summed (in $\vec{s}$) is the result applied to $\vec{w}$ to take a step. This guarantees that the step taken is in the downhill direction. Yet, for large datasets, this can be wasteful. Especially in the beginning of the optimization, when $\vec{w}$ is far from the optimal solution, many of the examples are misclassified and most of their per-item loss gradients point in a useful direction.

Therefore, we might consider updating $\vec{w}$ in the middle of the inner loop. Then, the per-item gradients calculated in subsequent

iterations would be from a (hopefully) better location. This results in Algorithm 5.11 (the line in gray is explained below).

This is an example of an **on-line learning algorithm**: It makes an update (learns) based on a single example. The perceptron learning algorithm is also an on-line learning algorithm. By contrast, the gradient descent method for logistic regression (Algorithm 5.10) is a **batch learning algorithm**: It requires all of the examples in the training set at the same time.

For large problems, where all of the data might not fit in memory at the same time, on-line learning is often preferred. If the data are streamed and not saved, on-line learning allows for continual updating of the learned function. However, judging convergence of an on-line method is more difficult. Parallelization of batch methods (to allow them to operate on multi-core, multi-processor, distributed, or vector-processor machines) is more straight-forward. Hybrids (called **mini-batch** methods) are popular, but beyond what we cover here.

Stochastic Gradient Descent

Algorithm 5.11 is an example of **stochastic gradient descent** (SGD), a variant of gradient descent that is applicable when the function to be optimized is a sum of subfunctions:

$$L(\vec{w}) = \sum_{i=1}^m l_i(\vec{w}) . \quad (5.16)$$

In our case $l_i(\vec{w}) = -\ln \sigma(y_i \vec{x}_i^T \vec{w})$, the per-item loss of example i . Stochastic gradient descent updates the vector ($\vec{w}$ in our case) based on the gradient of one of the subfunctions (rather than summing up all of their gradients before making an update).

The stochastic gradient descent algorithm that is usually analyzed picks the subfunction at random (hence the “stochastic” part of its name), instead of looping over the points in the same order. If the subfunctions (corresponding to data points) are chosen in the same order, the algorithm can get into cycles and never converge. However, it is sufficient to randomize the order of the points before each inner loop. Each update of $\vec{w}$ is called a step, and a full set of steps over each example in the training set is called an **epoch**. General stochastic gradient descent for loss minimization is shown in Algorithm 5.12.

Loss Functions

The perceptron learning algorithm (Algorithm 5.3) and the on-line logistic regression learning algorithm (Algorithm 5.11) are very similar. The perceptron algorithm does not randomize the order of the examples, but variants do. The perceptron algorithm does not have

ON-LINE LOGISTIC REGRESSION LEARNING

In: X, Y, η

Out: $\vec{w}$

$\vec{w} \leftarrow$ arbitrary point

while not converged **do**

 (shuffle order of training data)

for all $i \in \{1, \dots, m\}$ **do**

$\vec{w} \leftarrow \vec{w} + \eta (1 - \sigma(y_i \vec{x}_i^T \vec{w})) y_i \vec{x}_i$

return $\vec{w}$

Algorithm 5.11: On-line logistic regression learning algorithm

SGD LOSS MINIMIZATION

In: X, Y, η

Out: $\vec{w}$

$\vec{w} \leftarrow$ arbitrary point

while not converged **do**

 shuffle order of training data

for all $i \in \{1, \dots, m\}$ **do**

$\vec{w} \leftarrow \vec{w} - \eta \nabla l_i(\vec{w})$

return $\vec{w}$

Algorithm 5.12: Stochastic gradient descent algorithm for loss minimization.

a step size (η). However, it works fine with any positive step size, so we can add a step size. The only other difference is in the formula in the inner loop, but even this is very similar. The perceptron update has an if-statement. If we let $\theta(y\tilde{f})$ be 1 if $y\tilde{f} > 0$ and 0 otherwise, then $\theta(-y\tilde{f}(\vec{x}_i))$ is 1 if the update is taken and 0 if it is not. We can write the two updates as

$$\vec{w} \leftarrow \vec{w} + \eta \theta(-y\tilde{f}(\vec{x}_i)) y_i \vec{x}_i \quad \text{perceptron} \quad (5.17)$$

$$\vec{w} \leftarrow \vec{w} + \eta (1 - \sigma(y_i \tilde{f}(\vec{x}_i))) y_i \vec{x}_i. \quad \text{logistic regression} \quad (5.18)$$

In the perceptron case, the $\theta(-y\tilde{f}(\vec{x}_i))$ term allows the addition of $y_i \vec{x}_i$ to $\vec{w}$ when the example is incorrectly classified, and prevents any change when the example is correctly classified: a hard threshold on the update. For logistic regression, $y_i \vec{x}_i$ is always added to $\vec{w}$, but only partially. The $(1 - \sigma(y_i \tilde{f}(\vec{x}_i)))$ term is larger when the discriminant disagrees with the sign of y_i “more.” So, the logistic regression version makes a larger update the more incorrect the discriminant is.

While it was not originally developed this way, the perceptron learning algorithm is also an example of stochastic gradient descent. The gradients above come from the per-item losses below (plotted in Figure 5.13).

$$l(y, \tilde{f}) = -\theta(-y\tilde{f})y\tilde{f} \quad \text{perceptron} \quad (5.19)$$

$$l(y, \tilde{f}) = -\ln \sigma(y\tilde{f}) \quad \text{logistic regression} \quad (5.20)$$

As for many binary classification methods, the loss can be expressed in terms of the product of the label (y) and the discriminant ($\tilde{f}$), at least when $y \in \{-1, +1\}$. When this product is positive, the example is correctly classified. When it is negative, the example is incorrectly classified.

Example

Consider the binary classification task of determining whether a hippopotamus is aggressive. The collected data are as in Table 5.1. We take “yes” to be the positive class and include an initial “zeroth” constant feature to allow for an intercept.

Using a stepsize of $\eta = 0.05$, Figure 5.14 shows the convergence of Algorithm 5.10 on this dataset. With an adaptive stepsize, convergence would be faster, but we stick with a constant stepsize to match the algorithm. Table 5.15 shows the calculations for this first step. The weight vector starts (arbitrarily) as $[2 \ -1 \ 6]^T$. The table shows the calculation of $\vec{s}$ through the inner loop. The calculations after the table are the update of the weight vector based on these calculations.

Further information

$$\text{Step function: } \theta(v) = \begin{cases} 1, & \text{if } v > 0 \\ 0, & \text{otherwise} \end{cases}$$

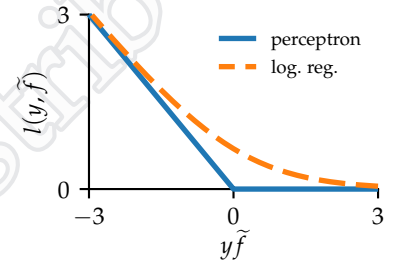


Figure 5.13: Perceptron and logistic regression per-item losses

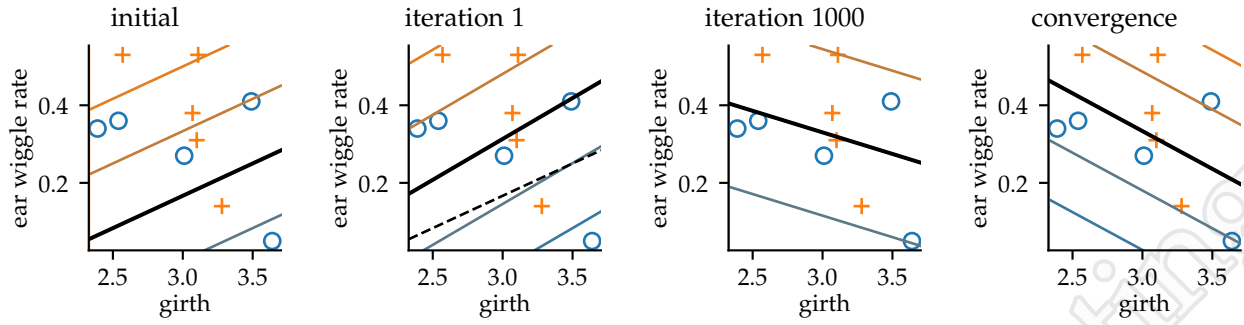


Figure 5.14: A few iterations of the batch logistic regression learning algorithm, starting with a non-zero w and using an intercept term. Data as in Table 5.1.

Orange +s are positive points. Blue Os are negative points.

Bold line is the decision boundary (other discriminant contours shown as in Figure 5.6).

Dashed line is the previous iteration's boundary.

$$\vec{w} = \begin{bmatrix} 2.00 & -1.00 & 6.00 \end{bmatrix}^T$$

i	$\vec{x}_i$	y_i	$\vec{x}_i^T \vec{w}$	$\sigma(y_i \vec{x}_i^T \vec{w})$	$(1 - \sigma(y_i \vec{x}_i^T \vec{w})) y_i \vec{x}_i$
1	$\begin{bmatrix} 1.00 & 3.49 & 0.41 \end{bmatrix}^T$	-1	0.97	0.27	$\begin{bmatrix} -0.73 & -2.53 & -0.30 \end{bmatrix}^T$
2	$\begin{bmatrix} 1.00 & 2.39 & 0.34 \end{bmatrix}^T$	-1	1.65	0.16	$\begin{bmatrix} -0.84 & -2.00 & -0.29 \end{bmatrix}^T$
3	$\begin{bmatrix} 1.00 & 3.28 & 0.14 \end{bmatrix}^T$	+1	-0.44	0.39	$\begin{bmatrix} 0.61 & 2.00 & 0.09 \end{bmatrix}^T$
4	$\begin{bmatrix} 1.00 & 3.10 & 0.31 \end{bmatrix}^T$	+1	0.76	0.68	$\begin{bmatrix} 0.32 & 0.99 & 0.10 \end{bmatrix}^T$
5	$\begin{bmatrix} 1.00 & 3.01 & 0.27 \end{bmatrix}^T$	-1	0.61	0.35	$\begin{bmatrix} -0.65 & -1.95 & -0.17 \end{bmatrix}^T$
6	$\begin{bmatrix} 1.00 & 3.11 & 0.53 \end{bmatrix}^T$	+1	2.07	0.89	$\begin{bmatrix} 0.11 & 0.35 & 0.06 \end{bmatrix}^T$
7	$\begin{bmatrix} 1.00 & 2.57 & 0.53 \end{bmatrix}^T$	+1	2.61	0.93	$\begin{bmatrix} 0.07 & 0.18 & 0.04 \end{bmatrix}^T$
8	$\begin{bmatrix} 1.00 & 3.07 & 0.38 \end{bmatrix}^T$	+1	1.21	0.77	$\begin{bmatrix} 0.23 & 0.71 & 0.09 \end{bmatrix}^T$
9	$\begin{bmatrix} 1.00 & 2.54 & 0.36 \end{bmatrix}^T$	-1	1.62	0.17	$\begin{bmatrix} -0.83 & -2.12 & -0.30 \end{bmatrix}^T$
10	$\begin{bmatrix} 1.00 & 3.64 & 0.05 \end{bmatrix}^T$	-1	-1.34	0.79	$\begin{bmatrix} -0.21 & -0.76 & -0.01 \end{bmatrix}^T$
				$\vec{s} =$	$\begin{bmatrix} -1.92 & -5.15 & -0.70 \end{bmatrix}^T$

$$\begin{aligned} \vec{w} &\leftarrow \vec{w} + \eta \begin{bmatrix} -1.92 & -5.15 & -0.70 \end{bmatrix}^T \\ &= \begin{bmatrix} 2.00 & -1.00 & 6.00 \end{bmatrix}^T + 0.05 \begin{bmatrix} -1.92 & -5.15 & -0.70 \end{bmatrix}^T \\ &= \begin{bmatrix} 1.90 & -1.26 & 5.96 \end{bmatrix}^T \end{aligned}$$

Table 5.15: Calculations for the first iteration of Algorithm 5.10

Exercises

Exercise 5.1

- If the probability of Amahle winning the hippopotamus race is 25%, what are the log-odds of Amahle winning the race?
- What if the probability is 75%?
- If the log-odds of Amahle winning the hippopotamus race are 0.4055, what is the probability she wins the race?
- What if the log-odds were -0.4055 ?
- Prove that the sum of the log-odds of Amahle winning the race and the log-odds of Amahle not winning the race is always 0, no matter what the probability of Amahle winning the race.

Exercise 5.2

You are trying to determine if a magic potion is poisonous or not. You have decided to use the perceptron learning algorithm for this, designating the positive class as "poison" and the negative class as "not poison."

You have measured two attributes about various potions: x_1 : the pH of the liquid (as it differs from the average of 7), and x_2 : the salinity of the liquid as a percentage (as it differs from sea water of 3.5%). You've also added a constant feature that is always 1: x_0 .

The data are shown in Table 5.16.

- Figure 5.17 shows the initial starting decision line for the perceptron. If the intercept term, w_0 , is 1, what are the other values in the starting weight vector?
- What is the resulting weight vector after the perceptron learning algorithm's first three steps (where it processes the first three examples, each once).
- For the final weight vector from above (after executing the first three iterations of the perceptron learning algorithm), where does the decision boundary intersect the axes?
- How will the algorithm proceed after these three steps?

constant	pH	salinity	poison
1	1	2	-1
1	0	-1	+1
1	1	1	-1

Table 5.16: Small potion dataset

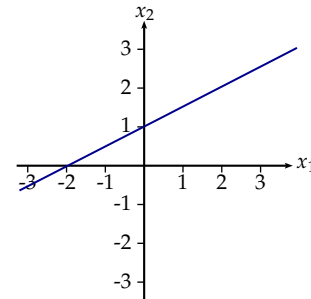


Figure 5.17: Starting decision boundary

Exercise 5.3

Write pseudo-code for performing gradient descent on the function

$$g(\vec{z}) = z_1^2 + e^{z_1 z_2} + e^{-z_2 z_3} + (z_3 - 2)^2.$$

Exercise 5.4

Consider the same example as Exercise 5.2 (data in Table 5.16), but

with logistic regression: Start with the same weight vector and use the same set of three training examples. Assume $\eta = 1$.

- What is the resulting weight vector after a single epoch (one pass through the three training examples) of online logistic regression learning?
- What is the resulting weight vector after one step of batch gradient descent logistic regression learning?
- In each of the two algorithms above, what will happen next?

Exercise 5.5

Prove that the logistic regression loss function, Equation 5.12, is convex in $\vec{w}$.

Exercise 5.6

Prove that if a binary classification dataset is separable with vector $\vec{w}$, then there is no minimum of the logistic loss function, Equation 5.12.

Exercise 5.7

Consider using the perceptron or logistic regression learning on the data in Table 5.16, but constraining the decision boundary to pass through the origin. To do so, we discard the “constant” feature.

- Plot the (total) loss as a function of the two weights for the logistic regression loss. Plot for the weights (each) ranging from -4 to $+4$.
- Make the same plot, but for the perceptron loss.

Now consider if the first element of the dataset had features of $[-1, -2]$ (instead of $[1, 2]$).

- Make the same plot for the logistic regression loss on this modified dataset.
- Make the same plot, but for the perceptron loss.

In the first case, the data are separable. In the second, they are not (if we restrict the decision boundary to pass through the origin).

- How does this difference manifest itself in the plots?
- Why will the perceptron algorithm not halt in the case of the non-separable data?